

SETOUT

AI 활용 기술 문서

무엇을 제약했고, 무엇을 기각했으며, 어떻게 검증했는가.

플레이 링크	https://keizxvzx.github.io/setout/
플레이 영상	https://www.youtube.com/watch?v=A-o8sApajPk
소스 코드	https://github.com/keizxvzx/setout
개발 기록	doc/AI_LOG.md (저장소에 그대로 있음)
제출	NHN NAN 2026 게임 AI 해커톤 사전 과제 · 제출물 4번
참가	개인

요약

AI 는 런타임의 부품이 아니라 설계 단계의 상대다

이 부분이 이 문서의 목적이기도 하다. 구조는 두 층이다.

개발 단계	설계 논의·구현·검증 스크립트 작성을 Claude Code (Claude Opus 5) 와 함께 했다
런타임	게임 안에서 도는 것은 그 설계의 결과물이다. 외부 모델을 호출하지 않는다

GitHub Pages 는 정적 호스팅이라 API 키를 둘 수 없고, 심사자에게 키 입력을 요구하는 것은 "링크 클릭만으로 브라우저에서 바로 플레이" 라는 제출 조건에 어긋난다. 그래서 런타임 호출은 1단계에서 배제했다.

대신 게임 안에는 생성 → 솔버 검증 → 실패 이유 판정 → 재설계 루프를 클라이언트 코드로 구현했다. 검증기는 이동 판정에 쓰는 BFS 를 그대로 재사용한다.

커밋 메시지에 Co-Authored-By: Claude 트레일러를 남겨 어느 커밋에 AI 가 관여했는지 커밋 단위로 추적할 수 있게 했다.

이 문서는 추린 것이다

작업하면서 그 자리에서 쓴 개발 기록이 저장소의 doc/AI_LOG.md 에 있다. 단계마다 그때의 설계 판단, AI 에 준 지시, 검증 결과가 남아 있다. 이 문서는 거기서 구조·지시·판단 기록만 추렸다.

AI 에 준 상시 지시 — 작업 내내 유지한 넷

1. 코드를 쓰기 전에 구조와 원리를 먼저 설명할 것. 설명이 납득되지 않으면 코드는 보지 않는다. 구현을 받는 것보다 설계를 검증하는 것이 먼저다.
2. 결정이 필요한 지점에서는 임의로 고르지 말고 선택지와 근거를 제시할 것. 되돌리기 비싼 결정(좌표계, 그리기 평면, 데이터 구조)은 반드시 사람이 고른다.
3. 눈으로 보고 "되는 것 같다" 로 넘어가지 말 것. 반드시 자동 검증 스크립트로 확인한 뒤에 진행한다.
4. 단계마다 커밋하고, 커밋 전에 확인받을 것. 커밋 메시지는 무엇을 했는지가 아니라 왜 그렇게 했는지를 쓴다.

AI 디렉터 — 구조

AI는 레벨을 만들지 않는다. 플레이어를 읽고, 그 사람이 피하는 것을 만든다.

솔버는 길 찾기가 아니다

이 판단에서 나머지가 전부 파생됐다.

`findPath` 는 이미 놓인 판 위에서 길을 찾는다. 그런데 목표는 빈 바닥 칸이고 플레이어는 판을 그려서 간다. 디렉터가 물어야 하는 것은 "길이 있는가" 가 아니라 "배급량 이하의 칸을 그려서 닿는 방법이 있는가" 다. 그래프는 그대로 쓰고 간선에 비용만 붙였다.

화면 자리 `s` 로 한 칸 옮길 때

- `s` 에 보이는 판 칸이 있다 → 0 (이미 있는 길)
- `s` 가 비었고 기둥이 안 잡혀 있다 → `INK_COST` (그려야 한다)
- 기둥이 잡혀 있다 → 통과 불가

가중치가 0과 2뿐이라 **0-1 BFS** 한 번으로 최소 잉크가 나온다. 뺄은 칸은 환급이 0 이므로 환급으로 이 값을 더 내릴 수 없다 — 진짜 하한이다.

벽이 공짜로 나왔다. 세 번째 줄의 결과다. 판을 `z` 만큼 올리면 화면에서는 물러나지만 기둥은 계속 잡혀 있어, 그 자리는 비어 보이는데 그릴 수도 설 수도 없다. 벽을 안 넣기로 확정해 뒤였는데 지형 도구는 이미 있었다. 설계한 것이 아니라 코드에서 유도한 성질이므로 검사 항목으로 확인한다.

검증 3항목이 두 숫자의 부등식이 된다

0-1 BFS 를 두 번 돌린다. 한 번은 전부, 한 번은 착시 간선(`a.z !== b.z`)을 막고.

계획서의 질문	실제 판정
풀리는가	<code>illusionCost !== null</code>
배급량 안에서 풀리는가	<code>illusionCost ≤ inkMax</code>
최소 해법이 착시를 요구하는가	<code>honestCost > inkMax</code>

셋째가 중심이다. 착시를 강제한다는 것은 배급량을 두 값 사이에 끼우는 것이고, 그 구간이 비면 어떤 배급량으로도 안 되므로 지형을 다시 설계하라는 신호가 자동으로 나온다.

실패 이유가 다음 수정을 지시한다

무작위 재추첨이면 절차적 생성이고, 이유가 방향을 정해야 에이전트다.

실패	뜻	다음 수정
UNSOLVABLE	목표에 닿는 길이 없다	목표를 당기거나 구조물을 하나 더
OVER_BUDGET	배급이 최소 해법보다 적다	배급을 <code>illusionCost</code> 로 올린다
ILLUSION_UNUSED	정직한 길이 배급 안에 있다	배급을 <code>honestCost</code> 미만으로 조인다
NO_BUDGET_WINDOW	조일 구간이 없다	지형 재설계
ILLUSION_FORCED	정직 의도인데 착시가 강제된다	배급을 늘리거나 <code>zMax</code> 를 조인다
TOO_SHALLOW	최소 해법이 너무 짧다	목표를 밀어낸다

배급을 0 에서 출발시킨다. 얼마를 줘야 하는지 미리 정해 두지 않고 첫 판정이 반드시 실패하게 만든 다음, 그 실패가 짚어 주는 최소값을 받는다. 지형이 바뀌면 배급이 저절로 따라오고, 배급표를 손으로 관리할 일이 없다.

난수는 시드를 고정한다. `Math.random` 을 쓰면 "400개 시드 전부 통과했다" 가 확인할 수 없는 주장이 되고, 디렉터가 왜 그 층을 냈는지도 그때 그랬다로만 남는다.

성향을 읽고 맞선다

읽은 성향	근거 지표	맞서는 방법	쥐는 손잡이
늘 다리로 민다	<code>inkSpent</code> ↑ <code>illusionSteps</code> ↓	잉크를 조여 착시를 강제	<code>budget</code> ∈ (<code>illusion</code> , <code>honest</code>)
착시만 노린다	<code>illusionSteps</code> 비율 ↑	정렬이 불가능한 지형	<code>zMax</code> → 0
망설인다	<code>redraws</code> ↑	배급을 최소 해법에 바짝 조인다	<code>budget</code> → <code>illusionCost</code>
아직 모르겠다	—	둘 다 되는데 착시가 싼 층	무엇을 고르는지 본다

넷째는 계획에 없었다. 왜 필요했는지는 다음 장에 있다.

파일

```
src/stage.js    현재 층의 손잡이 { inkMax, zMax, refund, start, goal, fixtures }
src/solver.js   순수. 0-1 BFS 최소 잉크. 재는 것만 한다
src/director.js 순수. 층 후보 생성 → 검증 3항목 → 실패 이유 → 재설계 루프
src/profile.js  순수. 지표 → 성향 → 지시(intent, zMax, depth)
src/session.js  층 전환 배선. 무엇을 남기고 무엇을 지우는가
```

뒤의 넷은 살아 있는 `board` 를 한 줄도 읽지 않는다. 그래야 디렉터가 **아직 놓지도 않은 배치**에 대고 "이게 풀리는가" 를 물어볼 수 있다. 4단계에서 `findPath` 를 순수 함수로 묶어 둔 것이 여기서 회수됐다.

계획과 다른 것 하나. 검증 3항목을 `solver.js` 가 아니라 `director.js` 에 두었다. 판정은 층의 의도를 알아야 하는데 `solver` 에 두면 `solver` → `director` → `solver` 로 순환한다. **솔버는 최소 잉크만 재고, 그 숫자로 무엇을 판단할지는 디렉터의 몫으로 남긴다.**

판단 기록 — 실물

각 시도의 { 시도번호, 실패코드, 한 줄 이유, 다음 수정 } 을 배열로 남긴다. 콘솔과 기록 문서에만 남기고 화면에는 띄우지 않는다 — 띄우면 플레이어가 자기가 읽히고 있다는 것을 알아 행동이 바뀌고, 그러면 읽은 성향이 성향이 아니게 된다.

배급을 0 에서 출발시키므로 첫 시도는 언제나 실패한다. 그 실패가 최소값을 짚어 주는 것이 설계다. 아래는 formatLog 출력 그대로이고, 성향 지표만 반대로 넣은 두 사람이다.

다리로 미는 사람 — 2층째

디렉터 - 시드 7, 3번 시도, 냈다

읽음: 1층째 - 총당 착시를 0.0번밖에 안 건넜다. 그리기만으로는 못 가는 층을 낸다. 배급의 98% 를 쓰고 망설임이 11% 다. 여유를 줄인다.

1. [OVER_BUDGET] 최소 해법 18 에 여유 4 을 얹으면 배급 0 를 넘는다. 딱 맞추면 한 칸도 헛디딜 수 없다. (착시 18 / 그리기 24, 배급 0)
→ 배급을 22 로 올린다.
2. [OK] 착시로 18, 그리기만으로는 24. 배급 22 안에서는 착시뿐이다.
3. [OK] 착시로 18, 그리기만으로는 24. 배급 22 안에서는 착시뿐이다.
→ 창 22~22 에서 22 을 골랐다.

낸 층: 배급 22, 상한 8, 시작 (13,13) → 목표 (10,4)

2번 꺾인 판 6칸을 z=2 로 올려 목표 옆에 세웠다. 3D 에서는 2칸 떨어져 있고 아무것도 달아 있지 않다.

착시만 노리는 사람 — 4층째

디렉터 - 시드 11, 3번 시도, 냈다

읽음: 3층째 - 총당 착시를 2.0번 건넜다. 맞춤 정렬이 없는 층을 낸다. 배급의 80% 를 쓰고 망설임이 28% 다. 여유를 줄인다.

1. [OVER_BUDGET] 정직한 최소 해법 28 에 여유 4 을 얹으면 배급 0 를 넘는다.
→ 배급을 32 로 올린다.
2. [OK] 그리기만으로 28 에 풀린다. 맞춤 정렬은 없다.
3. [OK] 그리기만으로 28 에 풀린다. 맞춤 정렬은 없다.
→ 창 32~42 에서 34 을 골랐다.

낸 층: 배급 34, 상한 0, 시작 (14,15) → 목표 (1,10)

높이 상한을 0 으로 조였다. 올릴 수 있는 판이 없으니 맞춤 정렬도 없다.

앞사람에게는 착시를 강제하는 층(상한 8, 이음매)이, 뒷사람에게는 정렬을 없앤 층(상한 0)이 갔다. 지표 말고 다른 입력은 없다.

지시가 결과를 바꾼 자리

요청을 그대로 실행하지 않고 목적을 되짚은 사례들이다.

검증이 설계 판단을 반증했다 (2단계)

절반 환급을 고르면서 "이러면 징검다리 이동이 막힌다" 고 판단했는데, 검증 스크립트가 이를 반증했다 — 직접 60칸 대 징검다리 110칸으로 **1.8배 유리했다**.

N 칸 건너기에서 직접은 $2N$, 징검다리는 $2N - R(N-k)$ 이므로 징검다리가 싸지 않으려면 $R \leq 0$ 이다. **환급률로는 막을 수 없다는 것이 계산으로 나왔고**, 조건을 "밟았는가/아닌가" 로 갈랐다. 검증의 목적은 코드가 의도대로 도는지 확인하는 것이지만, 때로는 **의도 자체가 틀렸다는 것**을 알려준다.

읽히지 않는 상태가 구현의 이탈을 드러냈다 (3단계)

`occupied` 집합이 쓰기만 하고 **읽히지 않는 것**을 발견해 따라가 보니, 판을 올린 자리에 두 번째 판을 그릴 수 있는 구멍이 있었다. 사람이 방금 고른 규칙이 막으려던 상황이 정확히 이것이었었는데 구현이 조용히 다른 규칙으로 바뀌어 있었다. 규칙을 정하는 것과 코드가 그 규칙대로 도는 것은 별개다.

측정 도구 자체가 틀릴 수 있다 (4·5단계)

검사가 실패했을 때 두 번 다 코드가 아니라 **측정 배치가 틀렸다**. 한 번은 뱀처럼 접은 복도에 지름길이 생겨 있었고, 한 번은 검사 배치에 화면 겹침이 있었다. 둘 다 측정하려던 것과 다른 것을 재고 있었다. 이후 **"실패를 보면 시나리오를 먼저 의심한다"** 를 규칙으로 삼고, "이 배치에 겹치는 화면 자리가 없다" 류의 전제 검사를 검증 앞에 넣었다.

요청을 그대로 구현하지 않았다 (6단계)

"착시만 노리는 사람한테 널 층은 상한을 1로 조이면 되지 않나?" 에 대해 200층을 실측했더니 **198층에서 착시가 더 쏘다**. 올려 둔 한 칸은 벽이면서 동시에 **공짜 징검다리**였다 — 도구를 뺏으려던 층이 도구를 나눠 주고 있었다.

상한을 **0** 으로 조여 고쳤다. 어렵게 만든 것이 아니라 **구조적으로 불가능하게 만든 것**이라 200층 전부 두 값이 같다.

검사를 요구했더니 설계 결함이 나왔다 (6단계)

"디렉터가 성향을 제대로 읽는지 가상 플레이어로 돌려 봐." — 맞서는 층만 내면 4층째부터 성향이 무엇이든 같은 순서가 나왔다. 착시를 강제하면 누구나 이음매를 건너고, 정렬을 없애면 아무도 못 건너다. **읽었다고 한 것이 사실은 자기가 직전에 낸 층이었다.**

세 번째 의도를 넣었다 — 둘 다 갈 수 있는데 착시가 싼 층이다. **여기서 싼 길을 골랐는지 비싼 길을 골랐는지가 디렉터가 만들지 않은 유일한 정보다.** 선택 말고는 아무것도 다르지 않은 두 사람을 10층씩 돌려 확인했더니 이제 정반대 층을 받는다.

눈으로 알아챈 것은 하나도 없다 (6단계 끝)

기록을 담은 뒤 여섯이 더 나왔는데 다섯은 오류조차 내지 않았고 게임은 화면상 멀쩡히 돌고 있었다.

무엇이 잡았나	무엇을
조건을 정리하다	리셋에 예고가 없다는 것
검사	깜박임 위상이 뒤집힌 것
측정	헛디딜 여유 2→4, 그리고 그 걱정이 틀렸다는 것
도구가 안 돌아서	배급 창 배선이 끊긴 것
요약 대신 실물로 실었더니	<code>TOO_SHALLOW</code> 문구가 값과 어긋난 것
실제로 띄워 봤더니	정적 서버가 클론에 없는 것, 캔버스 0×0

"눈으로 보고 '되는 것 같다' 로 넘어가지 않는다" 가 마지막까지 값을 한 자리다. 그리고 그 원칙이 검사만 뜻하는 것이 아니라는 것도 여기서 드러났다 — **쓰려던 도구가 안 돌거나, 요약 대신 실물을 싣거나, 실제로 한 번 띄워 보는 것도 같은 일을 한다.**

말로 고르지 않았다 (4·6단계)

색·문구·연출은 전부 **게임의 렌더 함수를 그대로 호출해** 실물 후보를 뽑아 놓고 사람이 골랐다. 캔버스 컨텍스트만 흉내 내고 `drawFloor` / `drawPlates` / `drawActor` 를 불러 들어온 그리기 명령을 받아 적는 방식이다. 그림을 따로 그리면 실물과 달라지지만, 이렇게 하면 화면에 나올 도형 그대로다.

깜박임처럼 정지 화면으로 판단할 수 없는 것은 실제로 깜박이는 미리보기를 만들어 보고 골랐다. **글로 색을 고르는 일이 한 번도 없었다.**

시 에 준 주요 지시

원문 그대로가 아니라 요지로 옮긴다. 각 지시가 실제로 결과를 바꾼 자리를 함께 적는다.

- "기획 정리 파일을 자세히 읽고, 단계별로 논의하면서 진행할 것." → 전체 구현을 한 번에 받지 않고 단계를 쪼갠 이유. 한 번에 받은 코드는 검증할 수 없고, 문제가 생겼을 때 어디가 원인인지도 알 수 없다.
- "코드를 쓰기 전에 구조와 원리를 먼저 설명할 것." → 그리기 평면을 `z=0` 으로 고정한 결정이 여기서 나왔다. 설명을 요구하지 않았다면 "커서가 닿은 표면에 그린다" 로 구현이 시작됐을 것이고, 레이캐스팅과 모호한 후보 선택 문제를 뒤늦게 만났을 것이다.
- "3·4·6단계 계획 정리해줘." (매 단계 같은 순서) → 코드가 아니라 계획을 요구했다. 3단계에서는 투영식을 먼저 풀어 `(x-z, y-z)` 한 줄에서 집기·착시 탐색 공간·깨질 코드 세 곳이 한꺼번에 나왔고, 6단계에서는 "솔버는 길찾기가 아니다" 가 나와 거기서 0-1 BFS·검증 3항목·실패 코드·재설계 루프가 전부 파생됐다.
- "눈으로 확인하지 말고 검증 스크립트로 확인할 것." → 검증 묶음 열아홉이 여기서 나왔다.

검증

작업 내내 지킨 원칙이 하나 있다. **눈으로 보고 "되는 것 같다" 로 넘어가지 않는다.** 좌표 변환은 틀려도 화면상 그럴듯해 보이고, 깊이 정렬은 겹치기 전까지 증상이 없으며, 잉크 회계는 한참 플레이해 봐야 어긋난 것이 드러난다.

```
node test/run.mjs
```

19개 묶음이 좌표계·잉크 회계·깊이 정렬·착시 연결·솔버·디렉터·HUD 를 확인한다. 외부 테스트 프레임워크를 쓰지 않는다 — 게임이 라이브러리를 하나도 안 쓰는데 검증만 의존성을 들이면 "설치 없이 실행"이라는 전제가 흐려진다.

디렉터 쪽만 봐도, 착시 층·정적 층을 각각 100개 시드에서 전부 내는지, 짝여 준 대로 고치면 그 실패가 다시 안 나는지, 통과한 층의 최소 해법이 **실제로 착시 간선을 쓰는지**를 확인한다 — 부등식이 참인 것과 뜻이 성립하는 것은 별개다.

렌더 교정 셋은 **고치기 전으로 되돌려 실패하는 것까지** 확인했다. 통과하는 검사는 통과할 뿐이고, 그 검사가 무엇을 잡는지는 깨뜨려 봐야 안다.

외부 에셋 / 오픈소스 출처

없음. 이미지·사운드·폰트·라이브러리를 일절 쓰지 않는다. 게임 전체가 순수 HTML + JavaScript + Canvas 2D 이며, 화면에 보이는 모든 것은 좌표 계산으로 그린다. 3D 엔진도 물리 엔진도 쓰지 않는다.

검증 스크립트와 개발용 정적 서버(`serve.mjs`)도 node 표준 모듈만 쓴다. `package.json` 이 없고 설치 단계도 빌드 단계도 없다.

이 제약은 라이선스 문제를 피하려는 소극적 이유만은 아니다. 등각 투영에서 착시가 성립하려면 화면상 픽셀이 정확히 맞아떨어져야 하는데, 범용 엔진의 카메라와 깊이 버퍼는 오히려 그 정확도를 방해한다. 직접 좌표를 계산하는 쪽이 이 게임에서는 더 정확하다.

실제로 띄워 확인했을 때 **외부 요청이 한 건도 없다는** 것도 같이 확인됐다 — 모듈 14개가 전부 자기 저장소에서 200 으로 뜨고 그 밖은 없다.

원본은 저장소에 있다

이 문서가 추린 원본은 `doc/AI_LOG.md` 다. 단계마다 그때의 설계 판단·지시·검증 결과가 작업하던 그 자리에 서 쓴 그대로 남아 있다.

https://github.com/keizxvzx/setout/blob/main/doc/AI_LOG.md